

Neumann-elvű gép egységei. CPU, adatút, utasítás-végrehajtás, utasítás- és processzorszintű párhuzamosság.
Korszerű számítógépek tervezési elvei. Példák RISC és CISC architektúrákra, jellemzőik

NEUMANN-ELVŰ GÉP EGYSÉGEI

- Neumann Jánost 1945-ben foglalta össze az EDVAC-ról szóló jelentésében, a modern számítógépek alapját képző elveket
- a Neumann-architektúra nem egy konkrét gép, hanem tervezési filozófia (mai gépek esetén is erre épül a filozófia)

Neumann-elvek röviden

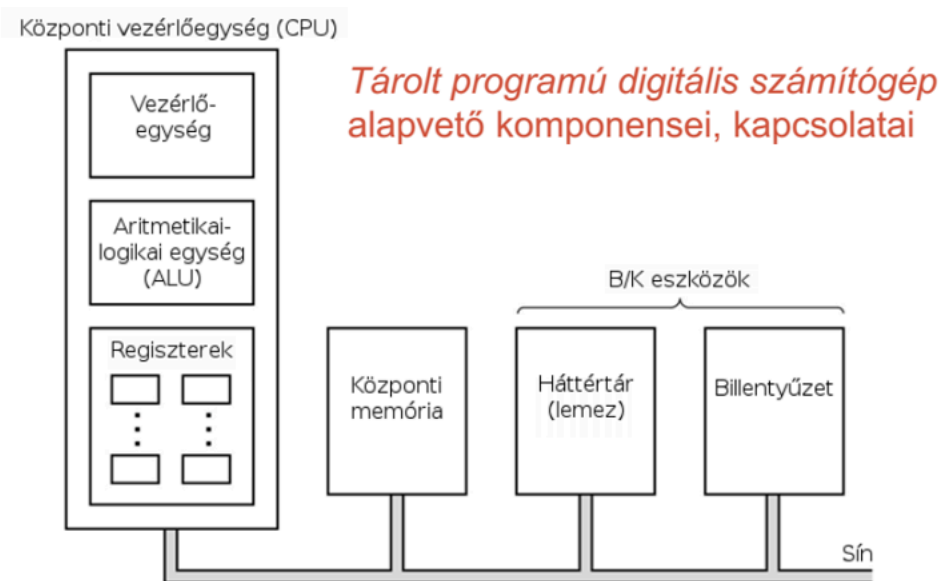
- **tárolt program elve**
 - legfontosabb újítás
 - az adatok és a programutasítások ugyanabban a belső gyorselérésű memóriában (tárban) helyezkednek el
 - kettes számrendszerben ábrázolva
- **kettes számrendszer alkalmazása**
 - a belső memóriában, mind az utasítások, mind az adatok bináris formában (bitek sorozataként) tárolódnak
- **soros utasítás-végrehajtás**
 - a gép az utasításokat egymás után, a memóriában tárolt sorrendbe hatja végre (hacsak egy ugróutasítás meg nem töri ezt)
- **univerzalitás**
 - a gép nem egy konkrét feladatra készül, hanem bármilyen algoritmus futtatására képes, ha a megfelelő utasítássorozatot betöltjük a memóriába

Neumann modell szerint a számítógép 5 fő részegységre bontható:

- **CPU (Központi Feldolgozó Egység)**
 - **ALU (Aritmetikai és Logikai Egység)**
 - a tényleges számításokat és logikai döntéseket végző áramkör
 - **Vezérlőegység (Control Unit - CU)**
 - értelmezi (dekódolja) az utasításokat és vezérli a többi egység munkáját (kapuzójeleket küld)
 - **Regiszterek**
 - processzor leggyorsabb, belső memóriái
- **Memória (tár)**
 - az adatok és az utasítások itt tárolódnak
- **Bemeneti egység**
 - kapcsolat a külvilággal, adatok bevitele
 - pl: billentyűzet, egér
- **Kimeneti egység**
 - az eredmények megjelenítése vagy továbbítása
 - pl: kijelző

- **Rendszersín (busz) - külső sín**

- az egységeket összekötő vezetékrendszer, amelyen adatok, címek és vezérlőjelek áramlanak
- **Neumann-szűk keresztmetszet**
 - mivel az adatok és az utasítások közös buszon (vezetéken) közlekednek a memória és a CPU között, a processzor sokszor várakozni kényszerül, mert a memória lassabb nála, ez korlátozza a gép sebességét
 - pl: Harvard-architektúra úgy védekezik ez ellen, hogy külön fizikai memória és sínrendszer (busz) van az utasításoknak és az adatoknak
 - ezzel egyszerre lehet olvasni mindkettőt



CPU, ADATÚT, UTASÍTÁS-VÉGREHAJTÁS, UTASÍTÁS- ÉS PROCESSZORSZINTŰ PÁRHUZAMOSSÁG

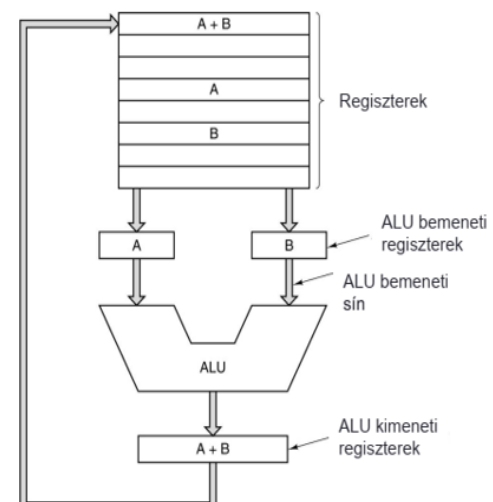
CPU (Központi Feldolgozó Egység)

- a CPU a számítógép agya, mely az utasítások értelmezéséért és végrehajtásáért felelős
- **fő részei**
 - **ALU (aritmetikai logikai egység)**
 - a tényleges számításokat és logikai döntéseket végző áramkör
 - azaz elvégzi az alapvető aritmetikai (összeadás, kivonás, léptetés) és logikai (ÉS, VAGY) műveleteket
 - **Vezérlő egység (CU - Control Unit)**
 - értelmezi (dekódolja) az utasításokat és vezérli a többi egység munkáját (kapuzójeleket küld, azaz nyitja vagy zárja a kapukat az egységek között)

- **Regiszterek**
 - processzor leggyorsabb, belső memóriái (cache)
 - **PC (Program Counter / Programszámláló)**
 - a következő végrehajtandó utasítás memóriacímét tartalmazza
 - HOL van a következő utasítás
 - **IR (Instruction Register / Utasításregiszter)**
 - az éppen végrehajtás alatt álló utasítás kódját tárolja
 - aktuális utasítás TARTALMA
 - **MAR (Memory Address Register)**
 - azt a címet tárolja, ahová írni akarunk, vagy ahonnan olvasni akarunk a memóriából
 - pufferként szolgál a CPU és külső rendszersín (busz) között
 - CPU és memória sebessége eltérő ezért kell ez, hogy az adatátvitel szabályozott legyen
 - **MDR (Memory Data Register)**
 - a memóriából kiolvasott vagy oda írandó adatot tárolja
 - pufferként szolgál a CPU és külső rendszersín (busz) között
 - CPU és memória sebessége eltérő ezért kell ez, hogy az adatátvitel szabályozott legyen
 - **Regisztertömb (GPR - General Purpose Registers)**
 - általános célú regiszterek az operandusok és részeredmények átmeneti tárolására (pl: R0, R1, EAX, stb)
- **Belső sín**
 - belső összeköttetés az ALU és regiszterek között

Adatút

- az adatút a CPU-n belüli úthálózat, amelyen az adatok mozognak a regiszterek és az ALU között
- feladata, hogy kiválasszon egy vagy két regisztert és az ALU-val műveletet végeztessen, majd ennek az eredményét egy regiszterbe írja
- **működése**
 1. a regiszterekből az adatok buszokon (síneken) keresztül az ALU bemenetére kerülnek
 2. az ALU elvégzi a műveletet, majd az eredmény egy buszon keresztül visszaíródik egy regiszterbe
- **adatút ciklus**
 - ez az az idő, ami alatt egy adat kijön a regiszterből, átmegy az ALU-n és az eredmény visszaíródik



Utasítás-végrehajtás folyamata

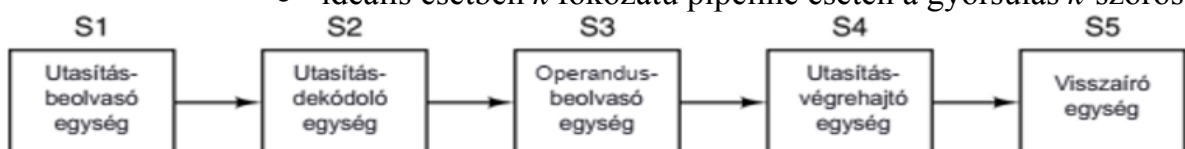
- bármilyen bonyolult egy program a CPU mindig ezt az ismétlődő ciklust folytatja (betöltő-dekódoló-végrehajtó ciklusnak is nevezik):
 1. **fetch (lehívás)**
 - a PC-ben (programszámláló regiszter) lévő memóriacím alapján az utasítás bekerül a memóriából az IR-be (utasításregiszter)
 - a PC-nek az értéke megnövekszik, azaz mutat a következő utasítás memóriacímére
 2. **decode (dekódolás)**
 - a vezérlőegység (CU) dekódolja az IR tartalmát
 - mit kell tenni? milyen adatokkal?
 3. **execute (végrehajtás)**
 - az adatút aktiválódik, az ALU elvégzi a műveletet, vagy adatmozgás történik
 4. **writeback (visszaírás)**
 - az eredmény beírása a regiszterbe vagy memóriába
- ugróutasítás (JUMP)
 - a fetch előtt a PC regiszter tartalmát az ugrási célcímmel felülírjuk

Párhuzamosság számítógépeken

- a modern számítógép teljesítményét már nem lehet pusztán órajel növeléssel fokozni a melegedés miatt, ezért a párhuzamosság felé fordultak a tervezők
- típus: **Utasításszintű párhuzamosság (ILP)** és **Processzorszintű párhuzamosság**

Utasításszintű párhuzamosság (ILP)

- klasszikus Neumann-gép egyszerre csak egy utasítást hajt végre ezért vezették be a **csővezeték (futószalag azaz pipelining) technikát**
 - aminek a teljesítmény növelése a célja
- **elve**
 - az utasítás-végrehajtási ciklust több (általában 5) részfeladatra (fokozat) bontjuk
 - **IF (Instruction Fetch):** utasítás lehívása a memóriából
 - **ID (Instruction Decode):** dekódolás és regiszterolvasás
 - **EX (Execute):** műveletvégzés az ALU-val vagy címformálás
 - **MEM (Memory Access):** adatmemória elérése (ha szükséges)
 - **WB (Write Back):** eredmény visszaírása a regiszterbe
 - amíg az egyik utasítás a második fázisban van (pl: dekódolás), addig a következő utasítás már bekerülhet az első fázisba (pl: lehívás)
 - ezzel egy utasítás végrehajtási ideje nem csökken, de az áteresztő képesség (időegység alatt végrehajtott utasítások száma) nő
 - pl: autómosó, egy autó ugyanannyi idő alatt ér végig, de percenként jönnek ki az új autók a végén
 - ideális esetben k fokozatú pipeline esetén a gyorsulás k -szoros

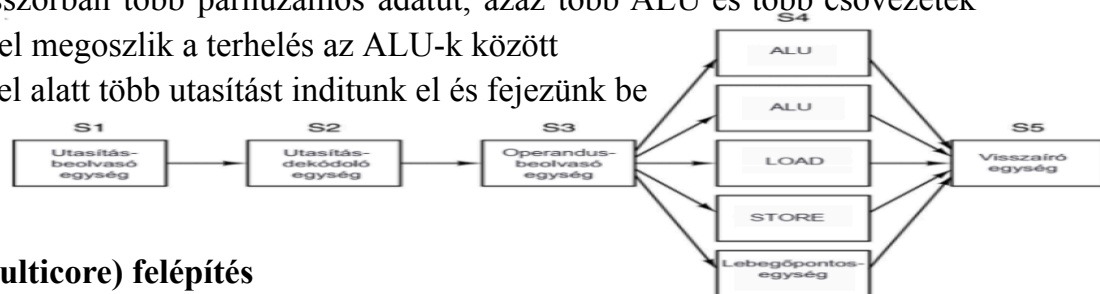


- a csővezetékekkel párhuzamosság nem mindig tökéletes, vannak gátló tényezők (akadályok):
 - **strukturális akadály**
 - két utasítás ugyanazt a hardveregységet akarná egyszerre használni, pl: ugyanazt a memóriaportot
 - **adatfüggőség (adatkonfliktus)**
 - egy utasítás olyan adatra vár, amit az előtte lévő még nem számolt ki
 - **vezérlési akadály**
 - egy feltételes ugrásnál nem tudjuk, melyik utasítás jön a csőbe, amíg el nem dől az ugrás iránya
 - megoldás rá az Ágvezérlés-jóslása
 - de ha ez rosszul tippel, jósol akkor "Pipeline flush" történik, azaz ki kell üríteni a csővezetékéből a már elindított, de hibás ágból származó utasításokat, és ez komoly idővesztés

Processzorszintű párhuzamosság

- **szuperskalár felépítés**

- a processzorban több párhuzamos adatút, azaz több ALU és több csővezeték van, ezzel megoszlik a terhelés az ALU-k között
- egy órajel alatt több utasítást indítunk el és fejezünk be



- **többmagos (multicore) felépítés**

- egyetlen szilíciumlapkára több teljes értékű processzormagot építenek (tömbprocesszor)
- itt már nem utasításokat, hanem különálló szálakat (threads) vagy programokat futtatunk párhuzamosan

- **multiprocesszorok**

- több teljes processzor van, amelyek közös memóriát használnak
 - ha szorosan együttműködnek akkor szorosan kapcsolataknak hívjuk
 - max pár száz processzort lehet összeépíteni
- legegyszerűbb, ha egyetlen sín van amihez a közös memóriát kapcsoljuk illetve a processzorokat
 - konfliktus veszélyes ha sok gyors processzor egyszerre akarja elérni a memóriát, ezért minden CPU-nak saját lokális memória is kell, amihez a többi nem fér hozzá

- **multiszámítógépek**

- sok összekapcsolt számítógép, lazán kapcsolt CPU-k
- nincs közös sín, a gépek hálózaton keresztül kapcsolódnak és a processzorok üzenet küldéssel kommunikálnak
- könnyen skálázható

KORSZERŰ SZÁMÍTÓGÉPEK TERVEZÉSI ELVEI

- a modern processzorok tervezésekor (kb 80-as évek) kialakult egy szabályrendszer, amely a hatékonyságot és a sebességet maximalizálja
- **Minden utasítás közvetlenül a hardveren hatódjon végre**
 - kerülni kell a mikroprogramozást (értelmezett kód, emuláció)
 - a hardveres vezérlés sokkal gyorsabb
- **Az utasítás-kiadási sebesség maximalizálása (párhuzamosítás)**
 - az utasítások kiadási üteme azt jelenti, hogy a processzor mennyi utasítást képes kiadni és végrehajtani egy adott időegység alatt
 - cél, hogy órajelenként minél több utasítás induljon el (párhuzamosítás, pipelining)
- **Könnyű dekódolhatóság**
 - az utasítások legyenek szabályosak, fix hosszúságúak és kevés mezőből álljanak
 - így a vezérlőegység (CU) gyorsan rájön, mit kell tennie
- **Load/Store architektúra**
 - csak speciális betöltő (Load) és tároló (Store) utasítások érhessek el a memóriát
 - minden más művelet (pl: összeadás) kizárólag regisztereken dolgozzon
 - egyszerűsíti a hardvert
- **Sok regiszter használata**
 - mivel a memória lassú, a CPU-ban minnél több gyors regiszternek kell lennie, hogy ott tartsuk az adatokat a számítások alatt

RISC ÉS CISC ARCHITEKTÚRÁK

- a processzorfejlesztés két fő iránya a CISC és a RISC, amelyek eltérő filozófiát követnek

CISC (Complex Instruction Set Computer)

- ez volt előbb
- összetett utasításkészletű számítógép
 - legyenek bonyolult, nagy tudású utasítások, hogy a programozó vagy a fordítóprogram dolga könnyű legyen
 - egyetlen utasítás elvégezhet egy komplex feladatot (pl. memóriából, olvas, összead, memóriába ír).
- **jellemzők**
 - sokféle utasítástípus
 - változó utasításhossz (1-től 15 bájtra nyúlhat)
 - sokféle címzési mód
 - kevés regiszter
 - bonyolult vezérlőegység (gyakran mikroprogramozott)
- pl: **Intel x86 család** (számítógépek nagy része) vagy a VAX
- a mai modern Intel (CISC) processzorok belül valójában RISC szerű mikorműveletekre bontják az összetett utasításokat, tehát kívülről CISC-nek látszanak, de belül RISC elven működnek

RISC (Reduced Instruction Set Computer)

- csökkentett utasításkészletű számítógép
 - csak egyszerű gyors utasítások legyenek, amik egy órajel alatt lefutnak (gyorsak)
 - a bonyolult műveleteket ezekből az egyszerű utasításokból rakja össze a szoftver (fordítóprogram)
- **jellemzők**
 - kevés utasítástípus
 - fix utasításhossz (pl: minden utasítás pontosan 32 bit)
 - **szabályos utasításformátum:** pl: az összeadás kódja az utasítás első 6 bitjén van, a forrásregiszter száma pedig mindig a következő 5 biten
 - így a hardvernek nem kell keresgélnie, hogy hol kezdődik az adat
 - Load/Store felépítés
 - sok regiszter, ezáltal kevesebb memória művelet
 - kiválóan párhuzamosítható (csővezetékbarát)
- pl: **ARM** (okostelefonok, Apple M1/M2/M3/M4), **MIPS**,
 - így a hardver is egyszerűbb lehet, ami alacsony fogyasztást és kisebb melegeledést is eredményezhet (hatékonyabbak)
 - ez mobilok és laptopok esetén kritikus